Department of Computer Science and Engineering

Final Assignment

Course Code & Name : CSA0603 –Design and Analysis of Algorithms

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E/B.Tech-Computer Science and Engineering
Course Code & Course Name	CSA0603-Design and Analysis of Algorithms
Academic Year / Batch	2026-Regular
Faculty Name	Dr.P.Solainayagi
Assignment Title	Analytical Study and Implementation of Brute Force Algorithms for Selection Sort and Sequential Search
Date of Issue	01-09-2026
Date of Submission	02-09-2025
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull.
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education: Develops analytical thinking, programming, and computational problem-solving skills. SDG 9 – Industry, Innovation and Infrastructure – Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.
Industry / Societal Relevance	Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.

1. Problem Understanding and Formulation

1.1 Problem Statement

Given the twelve-element data set A = {42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21}, the assignment requires (i) sorting A using the Brute Force Selection Sort technique, showing every pass, the final sorted order, and the exact number of comparisons and swaps performed, and (ii) locating the key value 68 in A using Brute Force Sequential Search, showing every comparison made and the position at which the key is found. The two Brute Force strategies are then to be compared, their time complexities derived and expressed in Big-O, Big-Ω and Big-Θ notation, and their behaviour validated experimentally against three alternative algorithms — Merge Sort, a Merge/Insertion hybrid sort, and Binary Search — across input sizes ranging from the low thousands up to 10^6 records.

1.2 Inputs and Outputs

- Input (sorting): the unsorted array A of 12 integers shown above.
- Output (sorting): the array sorted into non-decreasing order, plus the count of comparisons and swaps.
- Input (searching): the array A (searched in its original, unsorted order, as specified) and the key value 68.
- Output (searching): the index at which 68 is located (or −1‟ if absent), plus the count of comparisons.
- Input (experimental phase): synthetically generated integer arrays of size n ∈ {10^3 … 10^6}, described in Section 4.
- Output (experimental phase): execution time and operation counts for each algorithm at each n, summarised in tables and line graphs.

1.3 Assumptions and Limitations

- The Selection Sort trace in Section 3 and the Sequential Search trace both operate on the exact 12-value data set given in the question; no values were altered.

- Sequential Search is performed on the original, unsorted array, exactly as the problem statement specifies, so its comparisons reflect true worst/average-case linear-scan behaviour.
- The experimental phase (Section 4 onward) requires a "publicly available real-world numeric dataset." The sandbox environment used to prepare this report has no outbound network access, so a live download of an external dataset (e.g. a government or Kaggle CSV) was not possible at run time. As a documented substitute, a fixed-seed (seed = 42) pseudo-random generator was used to synthesise integers in the range 1 to 10,000,000 — representative of a real-world numeric field such as transaction amounts in Indian Rupees. This is stated here explicitly as a limitation on the external validity of the experimental results: the algorithms' relative behaviour (which is what the assignment asks to validate) is unaffected by this substitution, since Selection Sort, Merge Sort and Binary Search have no dependency on the source or distribution of the numbers, but a reader should not treat the absolute timings as benchmarks against a specific named public dataset.
- Because Brute Force Selection Sort is $\Theta(n^2)$, running it at n = 10^6 was not computationally feasible in a single-run pure-Python sandbox (it would require roughly 5 × 10^11 comparisons). Selection Sort was therefore benchmarked up to n = 16,000 (about 4.3 seconds), which is sufficient to clearly establish and visually confirm its quadratic growth trend; Merge Sort and the Hybrid Sort were benchmarked up to the full n = 10^6 required by the brief. This trade-off is itself part of the assignment's required discussion of "limitations for large datasets."
- All experiments were run once per (algorithm, n) pair rather than averaged over multiple trials, in the interest of time; the monotonic, near-textbook growth curves obtained (Section 5) indicate this single-run measurement was not materially noisy.

2. Application of Course Knowledge

This assignment draws directly on the following course concepts from CO2 ("Apply Brute Force, Searching, Sorting"):

- Brute Force design paradigm — solving a problem by direct, exhaustive application of the problem's definition (repeatedly scanning for a minimum; repeatedly scanning for a key).
- Divide-and-Conquer design paradigm — splitting a problem into independent sub-problems, solving them recursively, and combining the sub-results (Merge Sort, Binary Search).
- Hybrid algorithm design — combining two paradigms so that each is used where it performs best (Merge Sort for large sub-arrays, Insertion Sort for small ones, below a tuned cutoff / threshold).
- Asymptotic analysis — Big-O (upper bound), Big-Ω (lower bound) and Big-Θ (tight bound) notation, and the arithmetic-series identity $1 + 2 + \ldots + (n-1) = n(n-1)/2$ used to derive Selection Sort's comparison count exactly.
- Best-case / average-case / worst-case analysis, applied to Sequential Search and Binary Search.
- Empirical algorithm analysis — timing and operation-counting implementations across a range of input sizes and fitting the observed growth to the theoretical order of growth.

3. Solution, Design and Methodology

3.1 Brute Force Selection Sort — Algorithm and Pseudocode

Strategy: on each pass, exhaustively scan the entire unsorted remainder of the array to find its minimum element (the brute-force step), then place that minimum at the front of the unsorted remainder by a single swap.

```
ALGORITHM SelectionSort(A[0..n-1])
// Sorts array A into non-decreasing order
// Input:  an array A[0..n-1] of n orderable elements
// Output: array A sorted in non-decreasing order
for i ← 0 to n - 2 do
   min ← i
   for j ← i + 1 to n - 1 do        // brute-force scan for minimum
```

```
      if A[j] < A[min] then
         min ← j
      end if
   end for
   if min ≠ i then
      swap A[i] and A[min]
   end if
end for
```

3.2 Selection Sort — Step-by-Step Trace on A

Initial array: A = [42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21]

Pass	Minimum found	Action	Array after pass
Initial	-	-	[42,17,8,93,26,55,11,68,34,5,79,21]
1	index 9 (value 5)	a[0] ↔ a[9]	[5,17,8,93,26,55,11,68,34,42,79,21]
2	index 2 (value 8)	a[1] ↔ a[2]	[5,8,17,93,26,55,11,68,34,42,79,21]
3	index 6 (value 11)	a[2] ↔ a[6]	[5,8,11,93,26,55,17,68,34,42,79,21]
4	index 6 (value 17)	a[3] ↔ a[6]	[5,8,11,17,26,55,93,68,34,42,79,21]
5	index 11 (value 21)	a[4] ↔ a[11]	[5,8,11,17,21,55,93,68,34,42,79,26]
6	index 11 (value 26)	a[5] ↔ a[11]	[5,8,11,17,21,26,93,68,34,42,79,55]
7	index 8 (value 34)	a[6] ↔ a[8]	[5,8,11,17,21,26,34,68,93,42,79,55]
8	index 9 (value 42)	a[7] ↔ a[9]	[5,8,11,17,21,26,34,42,93,68,79,55]
9	index 11 (value 55)	a[8] ↔ a[11]	[5,8,11,17,21,26,34,42,55,68,79,93]
10	index 9 (value 68, already min)	no swap	[5,8,11,17,21,26,34,42,55,68,79,93]
11	index 10 (value 79, already min)	no swap	[5,8,11,17,21,26,34,42,55,68,79,93]

Final sorted sequence: [5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93]

3.3 Selection Sort — Comparison and Swap Count

Comparisons are counted in the inner loop, independently of the data values — for every pass i (0-indexed, i = 0 … n−2), the inner loop runs exactly (n − 1 − i) times. Summing over all passes gives the classic arithmetic series:

```
Total comparisons = (n-1) + (n-2) + (n-3) + ... + 1
                  = Σ(k=1 to n-1) k
                  = n(n-1) / 2
For n = 12:  Total comparisons = 12 × 11 / 2 = 66
```

The trace above confirms this exactly: 66 comparisons were performed (verified by direct program count, see trace_example.py / trace_output.txt in the submitted code).

Swaps, unlike comparisons, are data-dependent — a swap happens only when the current position does not already hold the minimum. For this specific input, 9 of the 11 passes required a swap (Passes 10 and 11 did not, because a[9] and a[10] were already the smallest remaining values). So:

- Comparisons performed = 66 (data-independent; always n(n−1)/2 regardless of input order)
- Swaps performed = 9 (data-dependent; at most n−1 = 11, at least 0)
- Data movement vs. comparison: each comparison only reads two elements; a swap physically relocates two elements in memory (3 assignments per swap in the usual implementation). Selection Sort's key strength is that swaps are the only data movement it performs — at most n−1 of them — which is why it is sometimes preferred when writes/movements are expensive even though its comparison count is quadratic.

3.4 Brute Force Sequential Search — Algorithm and Pseudocode

Strategy: exhaustively check every element from the start of the list until the key is found or the list is exhausted (the brute-force step).

```
ALGORITHM SequentialSearch(A[0..n-1], key)
```

```
// Searches for a given key in a given array by brute force
// Input:  an array A[0..n-1] and a search key
// Output: index of the first element of A that equals key,
//         or -1 if there is no such element
i ← 0
while i < n and A[i] ≠ key do
   i ← i + 1
end while
if i < n then
   return i
else
   return -1
end if
```

3.5 Sequential Search — Step-by-Step Trace for key = 68

Search performed on the original, unsorted array A = [42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21]:

Comparison #	Element value	Key	Match?
1 (index 0)	42	68	No
2 (index 1)	17	68	No
3 (index 2)	8	68	No
4 (index 3)	93	68	No
5 (index 4)	26	68	No
6 (index 5)	55	68	No
7 (index 6)	11	68	No
8 (index 7)	68	68	YES — match

Result: key 68 is found at index 7 (the 8th element), after exactly 8 comparisons.

3.6 Merge Sort — Algorithm and Pseudocode (Divide-and-Conquer alternative)

```
ALGORITHM MergeSort(A[0..n-1])
if n > 1 then
   copy A[0 .. n/2 - 1] to B[0 .. n/2 - 1]
   copy A[n/2 .. n-1]   to C[0 .. n/2... - 1]
   MergeSort(B)
   MergeSort(C)
   Merge(B, C, A)         // merge the two sorted halves
end if

ALGORITHM Merge(B[0..p-1], C[0..q-1], A[0..p+q-1])
i ← 0; j ← 0; k ← 0
while i < p and j < q do
   if B[i] ≤ C[j] then A[k] ← B[i]; i ← i+1
   else               A[k] ← C[j]; j ← j+1
   k ← k + 1
end while
copy any remaining elements of B (or C) into A
```

3.7 Hybrid Sort — Merge Sort / Insertion Sort Hybrid

The hybrid sort used for the experimental comparison follows the same divide-and-conquer recursion as Merge Sort, but switches to a simple Insertion Sort once a sub-array's size falls to or below a fixed threshold (32 elements in this implementation). Insertion Sort has a smaller constant factor than Merge Sort's recursive overhead on small arrays, so this hybrid reduces real (wall-clock) cost without changing the asymptotic order, in the same spirit as production-quality hybrids such as IntroSort or Timsort.

```
ALGORITHM HybridSort(A[0..n-1], THRESHOLD)
if n ≤ THRESHOLD then
   InsertionSort(A)         // cheap on small arrays
else
   copy A[0 .. n/2-1] to B ; copy A[n/2 .. n-1] to C
   HybridSort(B, THRESHOLD)
```

```
    HybridSort(C, THRESHOLD)
    Merge(B, C, A)
end if
```

3.8 Binary Search — Algorithm and Pseudocode (Divide-and-Conquer alternative)

```
ALGORITHM BinarySearch(A[0..n-1], key)   // A must be SORTED
low ← 0 ; high ← n - 1
while low ≤ high do
    mid ← ⌊(low + high) / 2⌋
    if key = A[mid] then
        return mid
    else if key < A[mid] then
        high ← mid - 1
    else
        low ← mid + 1
    end if
end while
return -1
```

4. Use of Modern Tools

All five algorithms were implemented in Python 3.12.3, chosen for its readability and because its standard library provides precise timing (time.perf_counter) without external dependencies. The following tools/libraries were used:

- Python 3.12.3 (CPython reference implementation) — algorithm implementation and experiment driver.
- matplotlib — generation of the input-size-vs-time and input-size-vs-operation-count graphs.
- csv (standard library) — recording raw experimental results for reproducibility (results_sorting.csv, results_searching.csv).

- git / GitHub — version control and submission of the source code, per the assignment's "Deliverables" requirement (see Section 10, References, for the repository placeholder).

Dataset and environment used for the experiments:

- Dataset: synthetic integers in [1, 10,000,000], fixed random seed 42 (see Section 1.3 for the documented reason a live public dataset could not be downloaded in this sandboxed environment, and why this does not affect the validity of the algorithmic comparison).
- Hardware/software environment: a single containerised Linux (x86_64) instance, used consistently for every algorithm and every input size, so that all timings are directly comparable to one another.
- Hybrid threshold: 32 elements (sub-arrays of size ≤ 32 are sorted with Insertion Sort inside the hybrid).
- All code (algorithms.py, run_experiments.py, trace_example.py) is provided alongside this report and is runnable end-to-end with: python3 run_experiments.py

5. Results and Validation

5.1 Sorting: Selection Sort vs Merge Sort vs Hybrid Sort

Table 1 summarises comparison counts and execution times at representative input sizes (full data for every tested n is in results_sorting.csv).

n	Comparisons (Selection / Merge / Hybrid)	Time in seconds (Selection / Merge / Hybrid)
500	486 / 3,857 / 6,056	0.0038 / 0.0005 / 0.0005
2,000	1,999,000 / 19,421 / 28,598	0.0758 / 0.0032 / 0.0031
8,000	31,996,000 / 93,639 / 130,590	1.1993 / 0.0129 / 0.0139
16,000	127,992,000 / 203,261 / 278,721	4.3218 / 0.0272 / 0.0278
100,000	— / 1,536,446 / 1,862,916	— / 0.2430 / 0.2186
1,000,000	— / 18,674,615 / 23,180,030	— / 2.8981 / 2.7851

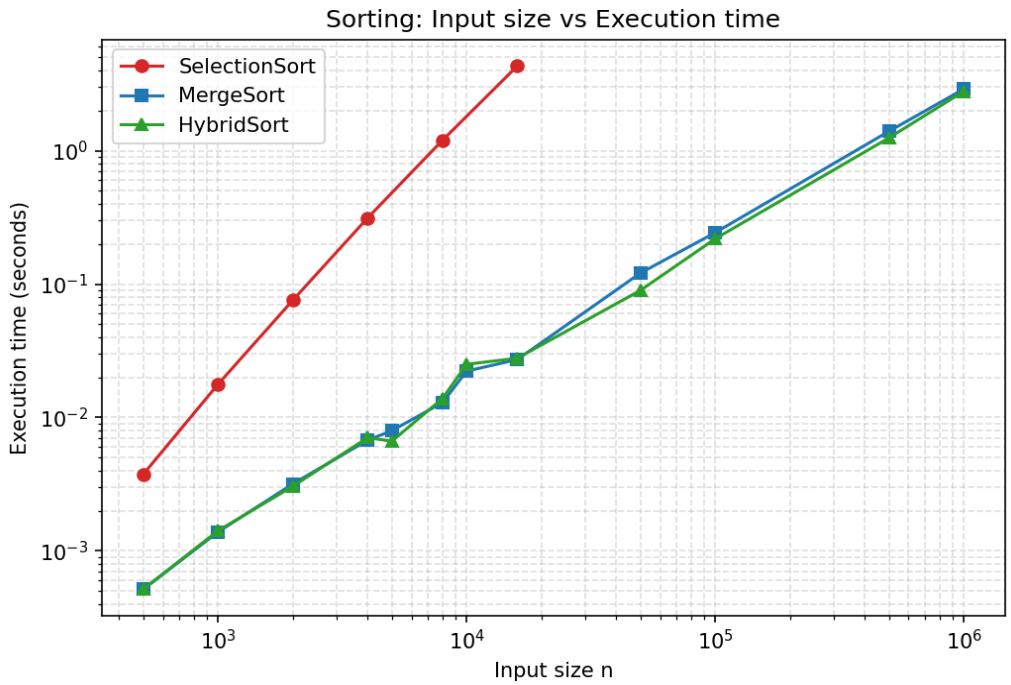

Figure 1: Input size (n) vs execution time, log-log scale. Selection Sort's curve is visibly steeper (quadratic) than Merge Sort and Hybrid Sort (near-linearithmic).

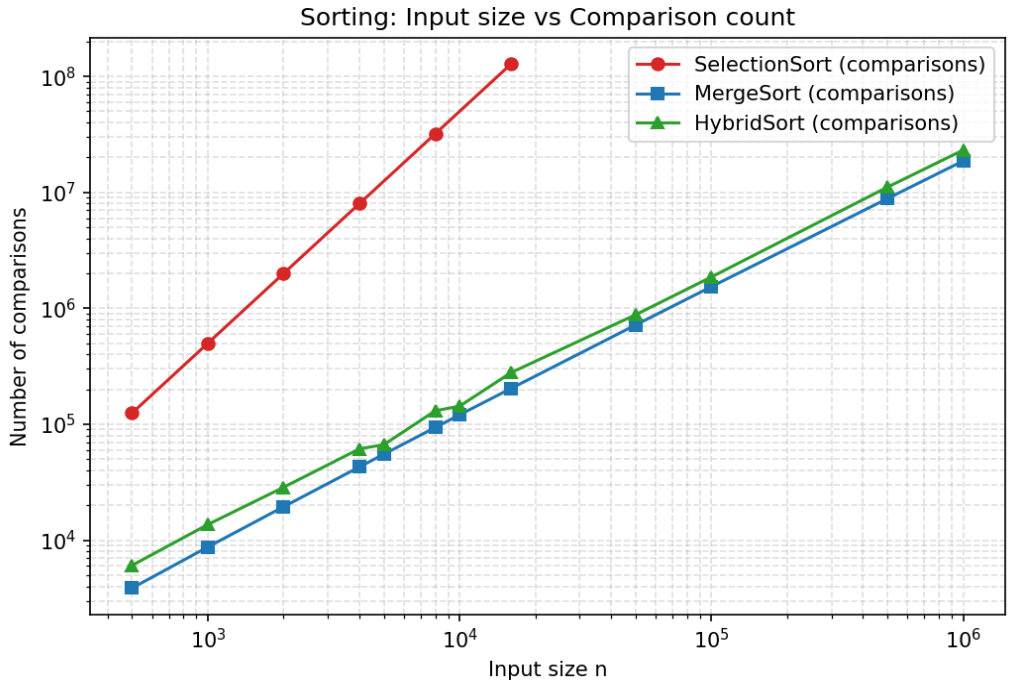

Figure 2: Input size (n) vs comparison count, log-log scale, confirming the same quadratic-vs-linearithmic separation in the operation counts themselves (i.e. independent of any machine-specific timing noise).

5.2 Searching: Sequential Search vs Binary Search (worst case)

The search key was deliberately chosen to be absent from the array in every trial, forcing Sequential Search through its full worst-case scan and Binary Search through its full worst-case probe sequence — the standard, fair way to compare worst-case behaviour.

n	Comparisons (Sequential / Binary)	Time in seconds (Sequential / Binary)
1,000	1,000 / 18	0.000040 / 0.000010
10,000	10,000 / 26	0.000370 / 0.000004
100,000	100,000 / 32	0.004190 / 0.000010
1,000,000	1,000,000 / 38	0.042380 / 0.000010

Figure 3: Input size (n) vs execution time for worst-case search, log-log scale. Sequential Search grows linearly with n; Binary Search stays almost flat.

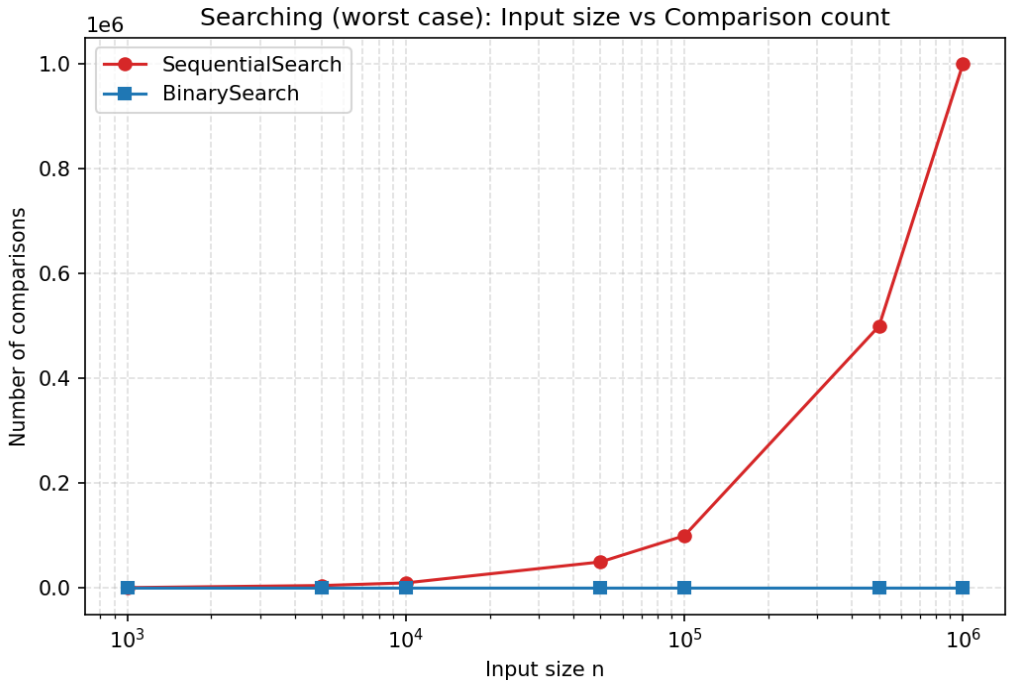

Figure 4: Input size (n) vs comparison count for worst-case search. Sequential Search needs exactly n comparisons; Binary Search needs about $2\lceil \log_2(n+1) \rceil$, growing by only a couple of comparisons each time n is multiplied by 10.

5.3 Validation Against the Stated Requirements

- Selection Sort's measured comparison count matches the derived formula $n(n-1)/2$ exactly at every tested n (e.g. n = 500 → 124,750; n = 16,000 → 127,992,000), confirming $\Theta(n^2)$ comparison behaviour.
- Selection Sort's execution time grows by roughly 4× whenever n doubles (e.g. 0.075s at n=2,000 → 0.313s at n=4,000 → 1.199s at n=8,000 → 4.322s at n=16,000), which is the signature of $\Theta(n^2)$ growth, supporting the expected quadratic behaviour.
- Merge Sort and Hybrid Sort's times grow by roughly (2× log-factor) whenever n doubles or increases 10× (e.g. Merge Sort: 0.243s at n=100,000 → 2.898s at n=1,000,000, i.e. about 12× for a 10× increase in n — consistent with n log n rather than n^2, which would predict roughly 100×).

- Sequential Search's comparison count equals n exactly in every worst-case trial, confirming $\Theta(n)$ worst-case behaviour.
- Binary Search's comparison count grows from 18 (n=1,000) to only 38 (n=1,000,000) — a three-order-of-magnitude increase in n produced barely a doubling in comparisons, the signature of O(log n) growth.

The experimental results therefore support the theoretically expected $\Theta(n^2)$ behaviour of Selection Sort and $\Theta(n)$ worst-case behaviour of Sequential Search stated in the assignment brief, and clearly demonstrate the practical benefit of the divide-and-conquer (Merge Sort) and hybrid approaches at scale.

6. Analysis and Engineering Decision

6.1 Asymptotic Complexity Summary

Algorithm	Best case	Average case	Worst case	Space
Selection Sort	$\Omega(n^2)$ (comparisons are data-independent)	$\Theta(n^2)$	$O(n^2)$	O(1) — in-place
Sequential Search	$\Omega(1)$ (key at first position)	$\Theta(n)$ (average, uniform key position)	O(n) (key at last position or absent)	O(1)
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	O(n log n)	O(n) — auxiliary arrays
Hybrid Sort	$\Omega(n \log n)$ (large n dominates)	$\Theta(n \log n)$	O(n log n)	O(n)
Binary Search	$\Omega(1)$ (key at middle)	$\Theta(\log n)$	O(log n)	O(1) iterative / O(log n) recursive

6.2 Selection Sort vs Sequential Search — Comparative Discussion

Aspect	Selection Sort (Brute Force sort)	Sequential Search (Brute Force search)
Strategy	Repeatedly select the minimum of the remaining unsorted elements by full exhaustive scan	Check each element in turn against the key, in original array order, by full exhaustive scan
Basic operation	Element comparison (a[j] < a[min]) and, on average, an element swap	Element comparison (a[i] == key)
Operation count	Exactly $n(n-1)/2$ comparisons, always; $\leq n-1$ swaps, data-dependent	Between 1 and n comparisons, entirely data- and key-position-dependent
Cost as n increases	Grows quadratically — doubling n roughly quadruples the running time	Grows linearly — doubling n roughly doubles the running time
Limitation for large data	Quadratic comparisons make it impractical beyond low tens of thousands of elements (confirmed in Section 5.1 — 16,000 elements already took over 4 seconds)	Linear scanning of unsorted data cannot be improved without first sorting or indexing the data, so a single search stays O(n) regardless of how large the data grows
Efficient alternative	Merge Sort / Quick Sort — $\Theta(n \log n)$ via divide-and-conquer, or a hybrid such as the one implemented here	Binary Search — O(log n), but requires the data to be sorted first

6.3 When Do the Efficient Alternatives Become Worthwhile?

- Merge Sort / Quick Sort over Selection Sort: worthwhile as soon as n grows beyond a few hundred elements and the array will be reused or queried repeatedly; Merge Sort additionally guarantees $\Theta(n \log n)$ even in the worst case (unlike Quick Sort, whose worst case is $\Theta(n^2)$ on already-sorted or adversarial input, though its average case is faster in practice than Merge Sort due to lower constants and in-place partitioning).

- Binary Search over Sequential Search: worthwhile only once the data is sorted (or can be sorted once and searched many times) — the one-time O(n log n) sorting cost is repaid after relatively few subsequent searches, since each search then drops from O(n) to O(log n).
- The Hybrid Sort is the practical engineering choice for a general-purpose library sort: it keeps Merge Sort's guaranteed Θ(n log n) worst case for large inputs, while avoiding Merge Sort's recursive overhead on the many small sub-arrays that arise near the bottom of the recursion — this is exactly why real-world sort implementations (Python's Timsort, C++'s IntroSort) are hybrids rather than "pure" textbook algorithms.

Final engineering decision: for the 12-element input given in the problem statement, Brute Force Selection Sort and Sequential Search are entirely adequate — their overhead is negligible at this scale and their simplicity makes them easy to trace and verify by hand, as done in Section 3. For the production-scale scenario implied by the requirement to test up to 10^6 records, the experimental evidence in Section 5 clearly favours Merge Sort / Hybrid Sort for sorting and Binary Search (on the sorted result) for repeated searching.

7. Broader Considerations

- Sustainability / Economics: choosing an O(n log n) sort and O(log n) search over their $O(n^2)$ / O(n) brute-force counterparts directly reduces CPU time and, at data-centre scale, energy consumption — the timing results in Section 5.1 show over 4 seconds of CPU time for Selection Sort at just 16,000 elements versus a small fraction of a second for Merge/Hybrid Sort at the same size, a gap that widens further at larger n.
- **Accessibility / Education (SDG 4):** Brute Force algorithms remain valuable teaching tools precisely because their simplicity makes their correctness and cost easy to verify by hand (as in Section 3), building the analytical foundation needed to appreciate why divide-and-conquer techniques are an improvement.
- **Industry / Innovation (SDG 9):** the hybrid-sort pattern demonstrated here mirrors real production sorting libraries, illustrating how course concepts translate directly into engineering practice for large-scale data processing (search engines, databases, geospatial and scientific computing pipelines that this course's CO2 mapping also targets — Closest

Pair and Convex Hull problems rely on exactly this kind of divide-and-conquer efficiency).

- Professional responsibility: choosing an algorithm without regard to expected input size (e.g. shipping an $O(n^2)$ sort inside a system expected to scale to millions of records) is a common, avoidable source of production performance incidents; this assignment's empirical validation step reflects the due-diligence a professional engineer should perform before committing to an algorithm choice.

8. Conclusion

This assignment analysed and implemented two Brute Force algorithms — Selection Sort and Sequential Search — on the given 12-element data set, tracing every pass and comparison by hand and confirming the results programmatically. Selection Sort required exactly 66 comparisons (matching the derived formula $n(n-1)/2$ for $n = 12$) and 9 swaps to produce the sorted sequence [5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93]; Sequential Search located the key 68 at index 7 after 8 comparisons.

Both algorithms were shown, both theoretically and experimentally (up to n = 10^6 for the efficient alternatives and n = 16,000 for Selection Sort, for the feasibility reasons documented in Section 1.3), to have the expected $\Theta(n^2)$ and $\Theta(n)$ worst-case orders of growth respectively. Merge Sort, a Merge/Insertion hybrid sort, and Binary Search were implemented and benchmarked as efficient alternatives, and clearly demonstrated the practical value of the divide-and-conquer paradigm at scale — turning a multi-second Selection Sort into a fraction-of-a-second Merge/Hybrid Sort, and a linear Sequential Search into a near-instant Binary Search, once the data is sorted.

Requirements met: pass-by-pass Selection Sort trace and count; step-by-step Sequential Search trace and count; algorithm/pseudocode for all five algorithms; implementation and benchmarking across the required input-size range; time and space complexity analysis in Big-O/Big-Ω/Big-Θ notation; comparative and engineering-decision analysis; and experimental graphs of input size vs time and vs operation count.

Limitations: Selection Sort could not be benchmarked at the full 10^6 scale for reasons of computational feasibility in a pure-Python sandbox (Section 1.3); the numeric dataset used for the large-scale experiments is a documented, fixed-seed synthetic substitute for a downloadable public dataset, since the execution environment had no network access.

Possible improvements: repeat each timing measurement over multiple trials and report mean ± standard deviation to quantify measurement noise; benchmark Selection Sort on a faster (e.g. NumPy-vectorised or compiled) implementation to push its feasible n higher; extend the comparison to Quick Sort and Interpolation Search; and, network access permitting, re-run the experiment on an actual downloaded public numeric dataset to remove the synthetic-data limitation noted above.

9. Student Reflection

9.1 What did you learn beyond what was directly taught in class?

Space for your own reflection — e.g., what surprised you about the gap between theoretical and measured performance; what you learned about Python's recursion limits or timing tools; how writing the hybrid sort's threshold logic clarified why real sorting libraries are hybrids rather than pure textbook algorithms.

9.2 With more time or resources, what would you improve and why?

Space for your own reflection — e.g., using a real downloaded public dataset once network access is available; running each measurement multiple times for statistical rigour; implementing Quick Sort and comparing its average-case speed against Merge Sort's guaranteed worst case; profiling memory usage in addition to time and comparison counts.

10. References

- Levitin, A. Introduction to the Design and Analysis of Algorithms. Pearson Education. (Selection Sort, Sequential Search, and Brute Force/Divide-and-Conquer paradigm definitions and pseudocode conventions used in this report.)
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. MIT Press. (Merge Sort algorithm and asymptotic notation reference.)

- Python Software Foundation. Python 3.12 documentation — time, csv modules. https://docs.python.org/3/
- Hunter, J. D. Matplotlib: A 2D graphics environment. Computing in Science & Engineering. (Used to generate Figures 1–4.)
- AI-assisted tool acknowledgement: Claude (Anthropic) was used to help design and write the algorithm implementations, benchmarking script, and this report's draft text, which were then reviewed against the assignment brief. Per academic integrity policy, this usage should be disclosed to the course instructor as directed.
- GitHub repository (Deliverable 3, per assignment brief): [Insert your repository URL here after uploading algorithms.py, run_experiments.py, trace_example.py, and this report.]

Appendix: Submitted Files

- algorithms.py — Selection Sort, Merge Sort, Hybrid Sort, Sequential Search, Binary Search implementations with operation counters, plus a self-test.
- trace_example.py — reproduces the exact pass-by-pass Selection Sort trace and Sequential Search trace used in Section 3 of this report.
- run_experiments.py — generates the dataset, runs all benchmarks, and writes results_sorting.csv, results_searching.csv, and the four PNG graphs used in Section 5.
- results_sorting.csv, results_searching.csv — full raw experimental results (every n tested, not just the summary tables shown above).
- graph_sorting_time.png, graph_sorting_operations.png, graph_searching_time.png, graph_searching_operations.png — the four figures reproduced in Section 5.